

Reflex Lab 01: Build Your First Guarded Reflex Circuit

Monogate Reflex Course First Live Lab Demo Packet

Threshold Reflex - Your First Verifiable EML Hardware Demo

****Status:**** Ready for your bench

****Goal:**** Run the Monogate loop on a safe trainer-board circuit and produce a trace that can be validated and replayed

Welcome!

Hi! Welcome to ****Reflex Lab 01****, your very first Monogate Reflex Course lab.

Today you're going to build a tiny but powerful circuit that shows the real magic of Monogate:

One clean EML kernel -> makes a guarded decision -> drives LED/button/buzzer I/O -> leaves a trace anyone can check

By the end you'll have:

- A working breadboard circuit
- Live kernel-controlled behavior you can see and tweak
- A verifiable trace you can replay and share
- A claim-bounded evidence packet starter

Let's get started!

What You'll See

This lab has three visible "wow" moments:

1. Turning the potentiometer changes the live dashboard value. 2. The LED follows the guarded output instead of blindly following the raw request. 3. When the request tries to go past the safe limit, the guard clamps it and the trace proves when it happened.

In Lab 01B, you add a button and buzzer. The button mutes the buzzer while the LED and guard behavior keep running, which makes the input/output boundary easy to feel.

Naming Map

Layer	Name
Platform	Monogate Electronics
Track	Reflex Lab Series
Board	Reflex Course
Course	Reflex Lab 01: Pot to Guarded Output
Kernel	<code>threshold_reflex_v0</code>

Learning Objectives

By the end of this lab you will be able to:

- Wire a safe ESP32 + potentiometer + LED/button/buzzer circuit on a breadboard
- Understand how `threshold_reflex_v0` controls visible breadboard outputs
- Capture and replay a live trace that shows whether the guard clamp fired
- Explain the Monogate way: input -> EML kernel -> guard decision -> visible output -> verifiable evidence

Safety First

Please read this before wiring:

- Use only 3.3V, never 5V, for this demo
- Always use the 220 ohm or 330 ohm resistor with the LED
- Disconnect USB before adding the Lab 01B button and buzzer stage
- Use the 1 kOhm resistor with the passive piezo buzzer stage
- Never connect the potentiometer to 5V or VIN
- If anything gets warm, unplug immediately
- Do not add motors, relays, solenoids, coils, or high-current parts yet

What You Need

Required for this first demo:

- ESP32 DevKit V1 or very similar
- USB data cable, not just a power cable
- Breadboard
- Jumper wires, male-to-male
- 10 kOhm potentiometer
- One LED, any color
- 220 ohm or 330 ohm LED resistor

- Tactile button
- Passive piezo buzzer
- 1 kOhm buzzer resistor
- Laptop with power

Nice-to-have for later demos, not needed today:

- SSD1306 OLED
- BME280 sensor

Software Setup: Do This First

Before touching wires, get the computer ready. This usually takes 10-20 minutes the first time.

Use the [Reflex Lab 01 Computer Setup Guide](setup-guide.pdf) for the full Windows | Mac path, preferred browser notes, serial-port examples, and troubleshooting.

1. Install Arduino IDE 2.x

Download Arduino IDE from:

```
https://www.arduino.cc/en/software
```

Install it normally.

Preferred browser for the dashboard path: Chrome or Edge. Safari is fine for reading docs, but it is not the supported path for direct browser serial features.

2. Add ESP32 Board Support

1. Open Arduino IDE. 2. Go to `File -> Preferences`. 3. In `Additional boards manager URLs`, paste:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

4. Go to `Tools -> Board -> Boards Manager`. 5. Search for `esp32`. 6. Install `esp32` by `Espressif Systems`.

3. Get The Firmware

The firmware is in the Monogate Electronics repository:

```
kernel/threshold_reflex_v0/esp32/threshold_reflex_v0/threshold_reflex_v0.ino
```

If you are using Git, clone or pull the repo. If you are not using Git, download the repository ZIP from GitHub and open that `.ino` file in Arduino IDE.

4. Check Your USB Cable

Use a USB data cable. A charge-only cable may power the ESP32 but will not show a serial port.

Before uploading firmware, confirm Arduino IDE can see your ESP32 port:

```
Tools -> Port
```

If no port appears, try another USB cable first.

Typical ports:

```
Windows: COM3, COM6, COM7
```

```
Mac: /dev/cu.usbserial-*, /dev/cu.SLAB_USBtoUART, /dev/cu.wchusbserial*
```

Mac instructions follow the standard ESP32/macOS workflow and are ready for students. Mark the Mac bench path as not yet Monogate bench-verified until a tester records a full Mac upload and dashboard run.

Optional: Python Trace Tools

Python is only needed for local trace validation and replay commands. If you want to run those checks on your laptop, install Python 3.10 or newer and confirm:

```
python --version
```

Step 1: Optional Computer Dry Run

Do this before you touch wires if you want the full Monogate validation path on your laptop. It proves the kernel and golden trace are healthy before hardware enters the loop.

Open your terminal or PowerShell in the `monogate-electronics` folder.

Validate the EML kernel:

```
python tools\validate_kernel_spec.py kernels\threshold_reflex_v0
```

Validate the golden trace:

```
python tools\validate_trace.py kernels\threshold_reflex_v0\traces\golden_trace.jsonl
```

Replay the golden trace:

```
python tools\replay_trace.py kernels\threshold_reflex_v0\traces\golden_trace.jsonl
```

You should see:

```
Smoke check PASS
```

or:

Trace validation: 7/7 valid

If anything fails, fix your Python setup before continuing.

Step 2: Build The Circuit On Breadboard

Quick Wiring Summary

For people who already know breadboards:

```
ESP32 GND -> blue/- rail
ESP32 3V3 -> red/+ rail
Pot outer leg 1 -> red/+ rail
Pot middle/wiper -> ESP32 GPIO34
Pot outer leg 2 -> blue/- rail
ESP32 GPIO25 -> 220/330 ohm resistor -> LED long leg
LED short leg -> blue/- rail
-- stop here for Reflex Lab 01A bring-up --
ESP32 GPIO27 -> tactile button
Button other side -> blue/- rail
ESP32 GPIO26 -> 1 kOhm resistor -> piezo +
Piezo - -> blue/- rail
```

Beginner Step-By-Step

2A. Connect Power Rails

Wire:

```
ESP32 GND pin -> breadboard blue/- rail
ESP32 3V3 pin -> breadboard red/+ rail
```

Check:

```
Red rail = 3.3V
Blue rail = GND
Red rail and blue rail are not shorted together
```

2B. Add The Potentiometer

Wire:

```
Left outer leg -> red/+ rail  
Middle leg/wiper -> ESP32 GPIO34  
Right outer leg -> blue/- rail
```

Check:

```
Turning the knob should change voltage safely between 0V and 3.3V.
```

If the knob feels backwards later, swap the two outside legs. Do not move the middle leg off GPIO34.

2C. Add The LED And Resistor

Wire:

```
ESP32 GPIO25 -> one end of resistor  
Other end of resistor -> LED long leg/anode  
LED short leg/cathode -> blue/- rail
```

Final safety check before power:

- Resistor is between GPIO25 and LED
- Potentiometer is only on 3.3V, GND, and GPIO34
- Nothing is warm
- No extra parts are connected yet

Take a photo of your finished breadboard.

2D. Stop, Program, Then Continue To Reflex Lab 01B

Do not add the buzzer yet. First power the 01A LED build, upload firmware, and confirm the dashboard reacts to the potentiometer.

After 01A works, disconnect USB. Lab 01B adds a digital button input and the audible buzzer output.

2E. Add The Button

Wire:

```
ESP32 GPIO27 -> tactile button GPIO side  
Button other side -> blue/- rail
```

Check:

```
The button shares ESP32 ground.  
No button leg is connected to 3V3, VIN, or 5V.
```

2F. Add The Buzzer And Resistor

Wire:

```
ESP32 GPIO26 -> one end of 1 kOhm resistor  
Other end of resistor -> piezo + leg  
Piezo - leg -> blue/- rail
```

Check:

```
The buzzer shares ground with the ESP32.  
The buzzer signal comes through the resistor.  
No buzzer lead is connected to VIN or 5V.
```

If your buzzer is very quiet, that is acceptable for this lab. The evidence claim is that the firmware reports and gates the audible output, not that the sound is loud.

Step 3: Upload The Firmware

The firmware file is already prepared for you:

```
kernels/threshold_reflex_v0/esp32/threshold_reflex_v0/threshold_reflex_v0.ino
```

Choose one method.

Option A: Arduino IDE

This is easiest for beginners.

1. Open the .ino file in Arduino IDE. 2. Select board:

```
ESP32 Dev Module
```

3. Select the correct USB port. 4. Click Verify first. 5. Click Upload. 6. Hold the BOOT button on some ESP32 boards if upload fails. 7. Open Serial Monitor at:

```
115200 baud
```

Option B: Arduino CLI

Use this if `arduino-cli` is installed.

Compile:

```
arduino-cli compile --fqbn esp32:esp32:esp32
kernels\threshold_reflex_v0\esp32\threshold_reflex_v0
```

Upload after checking the correct port:

```
arduino-cli upload -p COM3 --fqbn esp32:esp32:esp32
kernels\threshold_reflex_v0\esp32\threshold_reflex_v0
```

Open serial:

```
arduino-cli monitor -p COM3 -c baudrate=115200
```

Replace `COM3` with your actual ESP32 port.

Step 4: Power Up And Run The Demo

Plug in the USB cable.

Open the Serial Monitor at `115200` baud.

Slowly turn the potentiometer knob.

What you should see:

- When the knob is low, the LED is off or very dim
- The buzzer stays off when the guarded output is below the audible threshold
- As you turn through the threshold region, the LED gradually brightens
- When the guarded output is high enough, the buzzer turns on
- When the math would ask for too much output, the guard clamp kicks in

This clamp is the star of the show. It is the kernel limiting the requested output before the trainer-board adapter drives the LED.

The first demo path is:

```
potentiometer -> threshold_reflex_v0 -> guard clamp -> LED/buzzer -> serial trace ->
replay
```

Step 5: Capture, Validate, And Replay Your Trace

When you're happy with the demo, copy the live JSON output from Serial Monitor into a new file:

```
evidence/reflexcourse/first_reflex_demo_YYYY_MM_DD/live_trace.jsonl
```

Validate it:

```
python tools\validate_trace.py
evidence\reflexcourse\first_reflex_demo_YYYY_MM_DD\live_trace.jsonl
```

Replay it:

```
python tools\replay_trace.py
evidence\reflexcourse\first_reflex_demo_YYYY_MM_DD\live_trace.jsonl
```

Watch the LED behavior again on your screen.

Reflection And Evidence

Create a quick `FINDINGS.md` file in your evidence folder and answer:

- What did the guard clamp actually do?
- How did it feel different from normal Arduino code?
- What surprised you?

Save:

- Photos of the parts, power rails, pot, LED/resistor path, buzzer/button path, final powered breadboard, and dashboard capture
- Your live trace file
- Replay HTML exported from the dashboard
- Dashboard PNG or screenshot
- Validation output
- Replay output
- Any wiring notes

Use the [Evidence Packet Guide](evidence-packet-guide.md) for the full photo and tutorial checklist.

Run the packet manifest tool at the end:

```
python tools\make_packet.py evidence\reflexcourse\first_reflex_demo_YYYY_MM_DD
```

Troubleshooting

Stop immediately if:

- Anything gets warm
- ESP32 keeps resetting
- LED is wired without the resistor
- Potentiometer is connected to 5V

- You see smoke or notice strange smells

Unplug instantly if any stop condition appears.

Glossary

Term	What it means
ADC	Analog-to-Digital Converter. It turns knob voltage into a number.
GPIO	Programmable pin on the ESP32.
Wiper	Middle leg of the potentiometer.
Anode	Long leg of LED, positive side.
Cathode	Short leg of LED, ground side.
Guard clamp	The EML kernel's safety decision.

You Did It!

You just ran a guarded kernel through a physical trainer-board adapter and created a trace that can be validated and replayed.

This is the Monogate way.

Next steps, when you're ready:

- Add the buzzer or button
- Try other EML kernels
- Rebuild the same loop as a permanent trainer board in a future lesson

Drop your findings, photos, or questions in the Monogate group. I would love to see your demo.

Bench Notes

Use this page while building:

Date:

Builder:

ESP32 board type:

Any changes you made:

Photos taken:

Trace file saved as: